```
!pip install uv
!uv pip install pandas matplotlib numpy scipy scikit-learn
```

```
Collecting uv
  Downloading uv-0.10.8-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
Downloading uv-0.10.8-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (23.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.3/23.3 MB 38.8 MB/s eta 0:00:00
Installing collected packages: uv
Successfully installed uv-0.10.8
Using Python 3.12.12 environment at: /usr
Audited 5 packages in 130ms
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge
from urllib.request import urlretrieve

# Set random seed for reproducibility
np.random.seed(69) # nice
import os
```

```
def load_auto_mpg(sample_size=100, random_state=69):
    """
    Load Auto MPG dataset with specified sample size.
    Parameters:
    -----------
    sample_size : int or None
        Number of samples to randomly select from dataset.
        If None, use the full cleaned dataset.
    random_state : int
        Seed used for deterministic sampling.
    Returns:
    --------
    X : numpy array (n, 2) - features [weight, horsepower]
    y : numpy array (n, 1) - target [mpg]
    """
    # Load Auto MPG dataset (download only if missing locally)
    url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data'
    if not os.path.exists('auto-mpg.data'):
        urlretrieve(url, 'auto-mpg.data')

    column_names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model_year', 'origin', 'car_name']

    data = pd.read_csv('auto-mpg.data',
                   names=column_names,
                   sep=r'\s+',
                   na_values='?')
    data = data.dropna()

    # Sample specified number of records deterministically
    if sample_size is not None:
        sample_size = min(sample_size, len(data))
        data = data.sample(n=sample_size, random_state=random_state)

    # Extract features and target
    X = data[['weight', 'horsepower']].values
    y = data['mpg'].values.reshape(-1, 1)

    print(f"Dataset: {len(y)} cars")
    print(f"X shape: {X.shape}, y shape: {y.shape}")

    return X, y

def minMaxNormalize(T):
    T_norm = (T - np.min(T)) / (np.max(T)-np.min(T))
    return T_norm

X, y = load_auto_mpg(sample_size=None)

df_normalized = minMaxNormalize(X)

print(df_normalized)
```

```
[4.59167648e-01 7.45975658e-02]
```

```
 [4.59167648e-01 7.45975658e-03]
 [3.35492737e-01 2.35571260e-03]
 [3.59049863e-01 3.53356890e-03]
 [3.36474283e-01 2.74833137e-03]
 [3.96348645e-01 4.12249706e-03]
 [3.78680801e-01 3.72987829e-03]
 [3.93404005e-01 3.14095014e-03]
 [3.80643895e-01 4.31880644e-03]
 [4.25795053e-01 3.33725952e-03]
 [3.92422458e-01 3.72987829e-03]
 [4.58186101e-01 3.72987829e-03]
 [4.20887318e-01 5.49666274e-03]
 [4.24813506e-01 5.69297212e-03]
 [4.52296820e-01 5.69297212e-03]
 [5.04318806e-01 1.06007067e-02]
 [5.08244994e-01 5.49666274e-03]
 [6.25049077e-01 6.67451904e-03]
 [6.11307420e-01 5.88928151e-03]
 [5.60266981e-01 1.37416569e-02]
 [5.66156262e-01 1.45268944e-02]
 [6.61366313e-01 1.25638005e-02]
 [7.22222222e-01 1.15822536e-02]
 [5.91676482e-01 8.24499411e-03]
 [6.71181782e-01 7.65606596e-03]
 [5.02355713e-01 8.24499411e-03]
 [5.09226541e-01 8.24499411e-03]
 [4.61130742e-01 8.24499411e-03]
 [4.96466431e-01 7.65606596e-03]
 [4.86650962e-01 7.45975658e-03]
 [5.27875932e-01 8.63761288e-03]
 [5.53396152e-01 9.03023165e-03]
 [3.79662348e-01 5.49666274e-03]
 [3.88496270e-01 4.31880644e-03]
 [3.77699254e-01 4.31880644e-03]
 [4.08127208e-01 3.33725952e-03]
 [4.08127208e-01 4.71142521e-03]
 [4.14998037e-01 8.24499411e-03]
 [4.23831959e-01 5.69297212e-03]
 [4.31684335e-01 4.71142521e-03]
 [3.76717707e-01 4.12249706e-03]
 [3.76717707e-01 4.12249706e-03]
 [3.82606989e-01 4.12249706e-03]
 [5.69100903e-01 1.25638005e-02]
 [5.82842560e-01 7.65606596e-03]
 [4.98429525e-01 9.03023165e-03]
 [5.47506871e-01 1.29564193e-02]
 [5.14134276e-01 9.81546918e-03]
 [4.56223007e-01 7.45975658e-03]
 [5.70082450e-01 8.63761288e-03]
 [5.38672949e-01 7.85237534e-03]
 [4.09108755e-01 1.17785630e-03]
 [4.41499804e-01 7.45975658e-03]
 [5.06281900e-01 6.47820966e-03]
 [5.24931292e-01 7.06713781e-03]]
```

```
from sklearn.model_selection import train_test_split

def initial_split(X, y, test_size=0.2, random_seed=42):
    """
    Split data into development and test sets.
    Parameters:
    -----------
    X : numpy array (n, p)
    y : numpy array (n, 1)
    test_size : float (default 0.2)
    Proportion of data for test set
    random_seed : int (default 42)
    Random seed for reproducibility
    Returns:
    --------
    X_dev, y_dev : Development set (for training and CV)
    X_test, y_test : Test set (for final evaluation only)
    """
    X_dev, X_test, y_dev, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_seed
    )
    return X_dev, y_dev, X_test, y_test
```

```
from sklearn.preprocessing import PolynomialFeatures

def create_polynomial_features(X, degree):
    """
    Generate polynomial features up to specified degree.
    Parameters:
    -----------
    X : numpy array (n, p) - normalized features
    degree : int - polynomial degree
    Returns:
    --------
    X_poly : numpy array (n, m) - polynomial features (no bias column)
    Note: include_bias=False means no column of 1s is added
    """
```

```
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    return poly.fit_transform(X)
```

```
X_simple = np.array([[0.5, 0.3], [0.7, 0.6], [0.2, 0.9]])
X_simple_poly2 = create_polynomial_features(X_simple, 2)
X_simple_poly3 = create_polynomial_features(X_simple, 3)

print("Baseline: ")
print(X_simple)
print("Polynomial 2nd: ")
print(X_simple_poly2)
print("Polynomial 3rd: ")
print(X_simple_poly3)
```

```
Baseline:
[[0.5 0.3]
 [0.7 0.6]
 [0.2 0.9]]
Polynomial 2nd:
[[0.5  0.3  0.25 0.15 0.09]
 [0.7  0.6  0.49 0.42 0.36]
 [0.2  0.9  0.04 0.18 0.81]]
Polynomial 3rd:
[[0.5   0.3   0.25  0.15  0.09  0.125 0.075 0.045 0.027]
 [0.7   0.6   0.49  0.42  0.36  0.343 0.294 0.252 0.216]
 [0.2   0.9   0.04  0.18  0.81  0.008 0.036 0.162 0.729]]
```

```
def get_fold(X, y, fold_number, total_folds):
    """
    Retrieve a specific fold for k-fold cross-validation.
    Parameters:
    -----------
    X : numpy array (n, p)
    y : numpy array (n, 1)
    fold_number : int (0-indexed: 0, 1, ..., total_folds-1)
    total_folds : int
    Returns:
    --------
    X_train, y_train : All samples except fold_number
    X_val, y_val : Only samples in fold_number
    """
    # Split
    X_split = np.array_split(X, total_folds)
    y_split = np.array_split(y, total_folds)

    # Remove consumed
    X_val = X_split.pop(fold_number)
    y_val = y_split.pop(fold_number)

    # Merge non-consumed
    X_train = np.concatenate(X_split, axis=0)
    y_train = np.concatenate(y_split, axis=0)

    return X_train, y_train, X_val, y_val
```

```
X_test = np.arange(10).reshape(-1, 1)
y_test = np.arange(10).reshape(-1, 1)
for fold in range(3):
    X_train, y_train, X_val, y_val = get_fold(X_test, y_test, fold, 3)
    print(f"Fold {fold}: Val={y_val.flatten()}, Train={y_train.flatten()}")
```

```
Fold 0: Val=[0 1 2 3], Train=[4 5 6 7 8 9]
Fold 1: Val=[4 5 6], Train=[0 1 2 3 7 8 9]
Fold 2: Val=[7 8 9], Train=[0 1 2 3 4 5 6]
```

Problem 3.1: Understanding Regularized Gradients (15 points)

Part A: Verify Dimensions (3 pts)

1. What is the shape of e^T?

$(1, n)$

2. What is the shape of $e^T X$?

$(1, m)$

3. What is the shape of w?

$(1, m)$

4. Does the gradient have the correct shape?

Yes. In $rac\partial L\partial w = -rac2ne^T X + 2\lambda w$, each term has shape $(1, m)$, so the gradient is $(1, m)$.

Problem 3.2: Implement Regularized Gradient Descent (10 points)

Modify your VectorizedMLR from Assignment 4 to include L2 regularization.

Key changes:

1. Add `lambda_reg` parameter
2. Update gradient for w
3. Update loss

Testing:

- Verify that `lambda_reg=0` reproduces unregularized results
- Verify that `lambda_reg=0.001` converges and has smaller weights than the unregularized results
- Show that losses decrease over iterations
- *In your PDF, highlight the three modified lines with comments*

```
import numpy as np

class VectorizedMLR:
    """
    Multiple Linear Regression: ŷ = X@wᵀ + b

    Works for ANY number of features:
        - p = 1: Simple Linear Regression
        - p > 1: Multiple Linear Regression

    REQUIRED Dimensions:
        X: (n, p) - n observations, p features
        w: (1, p) - 1 row, p features (ROW VECTOR)
        b: scalar
        ŷ: (n,1) - n predictions
    """

    def __init__(self, learning_rate=0.01, num_iterations=1000, lambda_reg=0.01):
        self.learning_rate = learning_rate
        self.num_iterations = num_iterations
        self.w = None  # Will be (1, p)
        self.b = 0.0
        self.loss_history = []
        self.lambda_reg = lambda_reg # Initialize Lambda Rate ------------- Changed

    def predict(self, X):
        """
        Matrix-based prediction: ŷ = X@wᵀ + b

        Input:
            X: numpy array (n, p)
        Returns:
            y_pred: numpy array (n,1) - predictions
        """
        y_pred = X @ self.w.T + self.b
        return y_pred

    def compute_loss(self, X, y):
        """Mean Squared Error: (1/n) * sum((y_pred - y)^2)"""
        n = X.shape[0]
        y_pred = self.predict(X)
        loss = (1 / n) * np.sum((y_pred - y) ** 2) + self.lambda_reg * np.sum(self.w ** 2) # Added Lambda Regularization Term
        return loss

    def compute_gradients(self, X, y):
        """
        Computes gradients for w and b.
        grad_w = dJ/dw
        grad_b = dJ/db
        """
        n = X.shape[0]
        y_pred = self.predict(X)
        error = y_pred - y # Shape (n, 1)

        # Gradient w:
        # Shape must be (1, p) to match self.w
        # Formula: (2/n) * error.T @ X
        # (1, n) @ (n, p) -> (1, p)
        grad_w = (2 / n) * (error.T @ X) + 2 * self.lambda_reg * self.w # ------ Added Lambda Regularization Term

        # Gradient b:
        # Scalar mean of errors * 2
        grad_b = (2 / n) * np.sum(error)

        return grad_w, grad_b

    def fit(self, X, y):
        """
        Apply gradient descent to optimize the parameters of the model.

        Input:
            X: numpy array (n, p)
            y: numpy array (n, 1)
```

```
        Returns:
            self for chained calls.
        """
        n_samples, n_features = X.shape
        self.w = np.zeros((1, n_features))
        self.b = 0.0
        self.loss_history = []

        # Gradient Descent Loop
        for i in range(self.num_iterations):
            # Calculate gradients
            grad_w, grad_b = self.compute_gradients(X, y)

            # Update parameters
            self.w = self.w - (self.learning_rate * grad_w)
            self.b = self.b - (self.learning_rate * grad_b)

            # Track loss
            current_loss = self.compute_loss(X, y)
            self.loss_history.append(current_loss)

        return self
```

Problem 3.3: Build and Test Baseline Model (10 points)

Report:

Metric	Development Set	Test Set
MSE	18.3410	18.3461
R^2	0.6974	0.6998

Analysis (2-3 sentences):

Based on the results, we're seeing very good performance with no overfitting. Shockingly, we can see that there is *better* performance in the "Test Set" than the "Development Set".

```
# Data loaded in earlier cell
X, y = load_auto_mpg()

# Split the data between train and test sets
X_train, y_train, X_test, y_test = initial_split(X, y, test_size=0.2, random_seed=42)
```

```
Dataset: 100 cars
X shape: (100, 2), y shape: (100, 1)
```

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

# Fit on train set and transform train set
X_train_normalized = scaler.fit_transform(X_train)

# only transform the test set
X_test_normalized = scaler.transform(X_test)
```

```
# Initialize model with no lambda regularization
model = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=0)

# Train the model on the normalized development data
model.fit(X_train_normalized, y_train)
```

```
<__main__.VectorizedMLR at 0x7b3f96bc5e20>
```

```
from sklearn.metrics import mean_squared_error, r2_score

# Predict on Development Set
y_dev_pred = model.predict(X_train_normalized)
dev_mse = mean_squared_error(y_train, y_dev_pred)
dev_r2 = r2_score(y_train, y_dev_pred)

# Predict on Test Set
y_test_pred = model.predict(X_test_normalized)
test_mse = mean_squared_error(y_test, y_test_pred)
test_r2 = r2_score(y_test, y_test_pred)

print(f"Dev MSE: {dev_mse:.4f}, Dev R2: {dev_r2:.4f}")
print(f"Test MSE: {test_mse:.4f}, Test R2: {test_r2:.4f}")
```

```
Dev MSE: 14.1843, Dev R2: 0.7448
Test MSE: 10.8202, Test R2: 0.7146
```

Problem 3.4: Cross-Validation for Different Configurations (30 points)

Part A (n=100, degree=10): Optimal $\lambda = 0$, Training MSE $= 20.4853$, Validation MSE $= 20.8997$.

Part B (n=100, degree=15): Optimal $\lambda = 0.01$, Training MSE $= 21.2814$, Validation MSE $= 22.01299$.

Part C (n=200, degree=10): Optimal $\lambda = 0$, Training MSE $= 20.4853$, Validation MSE $= 20.8997$.

```
# Load data and split (sample_size=100, degree=10)
X, y = load_auto_mpg(sample_size=100)
X_dev, y_dev, X_test, y_test = initial_split(X, y, test_size=0.2, random_seed=42)

lambdas = [0, 0.0001, 0.001, 0.01, 0.1, 1, 10, 100]
degree = 10
total_folds = 5

# Arrays to store average MSE for each lambda
train_mse_averages = []
val_mse_averages = []
```

```
Dataset: 100 cars
X shape: (100, 2), y shape: (100, 1)
```

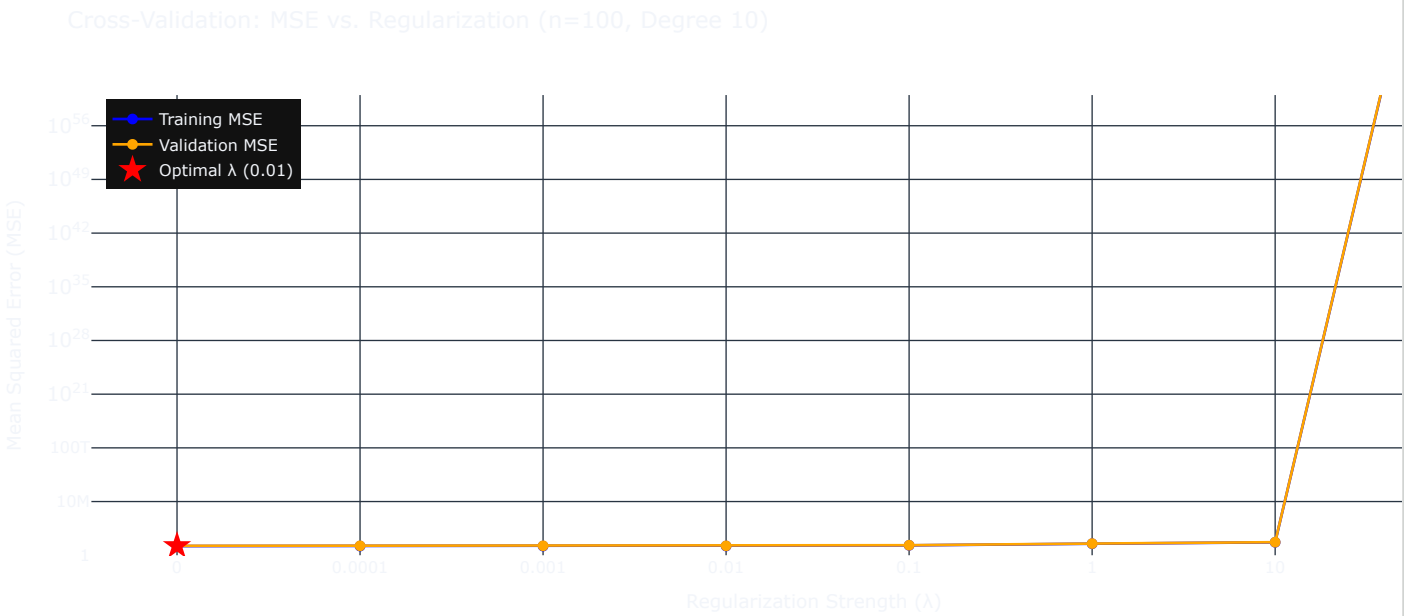
```
for lam in lambdas:
    fold_train_mses = []
    fold_val_mses = []

    for fold in range(total_folds):
        # Get the current fold
        X_train, y_train, X_val, y_val = get_fold(X_dev, y_dev, fold, total_folds)

        # Normalize (Fit ONLY on training data!)
        scaler = MinMaxScaler()
        X_train_norm = scaler.fit_transform(X_train)
        X_val_norm = scaler.transform(X_val)

        # Polynomial Expansion (MUST be after normalization)
        X_train_poly = create_polynomial_features(X_train_norm, degree)
        X_val_poly = create_polynomial_features(X_val_norm, degree)

        # Train the Model
        model = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=lam)
        model.fit(X_train_poly, y_train)

        # Predict and Calculate MSE
        y_train_pred = model.predict(X_train_poly)
        y_val_pred = model.predict(X_val_poly)

        fold_train_mses.append(mean_squared_error(y_train, y_train_pred))
        fold_val_mses.append(mean_squared_error(y_val, y_val_pred))

    # Calculate the average MSE across all 5 folds for this specific lambda
    train_mse_averages.append(np.mean(fold_train_mses))
    val_mse_averages.append(np.mean(fold_val_mses))
```

```
# Find index of the lowest validation MSE
best_index = np.argmin(val_mse_averages) #argmin finds smallest value in array and returns index
optimal_lambda = lambdas[best_index]

# For plotting purposes, scale is too big so we will set a max y value to zoom in
y_max = 60


# All MSEs for each lambda
print("Lambda\tTrain MSE\tVal MSE")
for i in range(len(lambdas)):
    print(f"{lambdas[i]}\t{train_mse_averages[i]}\t{val_mse_averages[i]}")


# Print the optimal lambda and corresponding MSEs
print(f"Optimal Lambda: {optimal_lambda}")
print(f"Training MSE at optimal lambda: {train_mse_averages[best_index]}")
print(f"Validation MSE at optimal lambda: {val_mse_averages[best_index]}")
```

```
Lambda  Train MSE       Val MSE
0       15.5430888161921        16.989191813157824
0.0001  15.544244402698235       16.987004669500532
0.001   15.555522265681129       16.96885748460202
0.01    15.742022437728156       16.917750017248377
0.1     19.41106677316937        20.20256380309105
1       31.724159908184042       33.120341175750525
10      48.53193746022943        50.16121640808365
100     6.384525850855931e+102  7.764119704185786e+102
Optimal Lambda: 0.01
Training MSE at optimal lambda: 15.742022437728156
Validation MSE at optimal lambda: 16.917750017248377
```

```
import plotly.graph_objects as go
# Handle lambda=0 for the log scale by shifting it to a tiny number
plot_lambdas = lambdas.copy()
```

```
plot_lambdas[0] = 0.00001
optimal_plot_lambda = 0.00001

fig = go.Figure()

# Add Training MSE Line
fig.add_trace(go.Scatter(
    x=plot_lambdas,
    y=train_mse_averages,
    mode='lines+markers',
    name='Training MSE',
    line=dict(color='blue', width=2),
    marker=dict(size=8)
))

# Add Validation MSE Line
fig.add_trace(go.Scatter(
    x=plot_lambdas,
    y=val_mse_averages,
    mode='lines+markers',
    name='Validation MSE',
    line=dict(color='orange', width=2),
    marker=dict(size=8)
))

# Add a big red star for the Optimal Lambda
fig.add_trace(go.Scatter(
    x=[optimal_plot_lambda],
    y=[val_mse_averages[best_index]],
    mode='markers',
    name=f'Optimal λ ({optimal_lambda})',
    marker=dict(color='red', size=16, symbol='star'),
    showlegend=True
))

# Format the layout, titles, and log scale
fig.update_layout(
    title='Cross-Validation: MSE vs. Regularization (n=100, Degree 10)',
    xaxis_title='Regularization Strength (λ)',
    yaxis_title='Mean Squared Error (MSE)',
    xaxis_type='log', # Set x-axis to log scale
    yaxis_type='log', # Set y-axis to log scale
    yaxis=dict(
        range=[0, y_max]  # <--- ADD THIS to force the zoom level
    ),

    # Customize the x-axis ticks so the '1e-5' point actually displays '0'
    xaxis=dict(
        tickmode='array',
        tickvals=plot_lambdas,
        ticktext=['0' if lam == 0 else str(lam) for lam in lambdas]
    ),

    template='plotly_dark',
    hovermode='x unified', # Shows all values when you hover over a specific lambda
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.01
    )
)

fig.show()
```



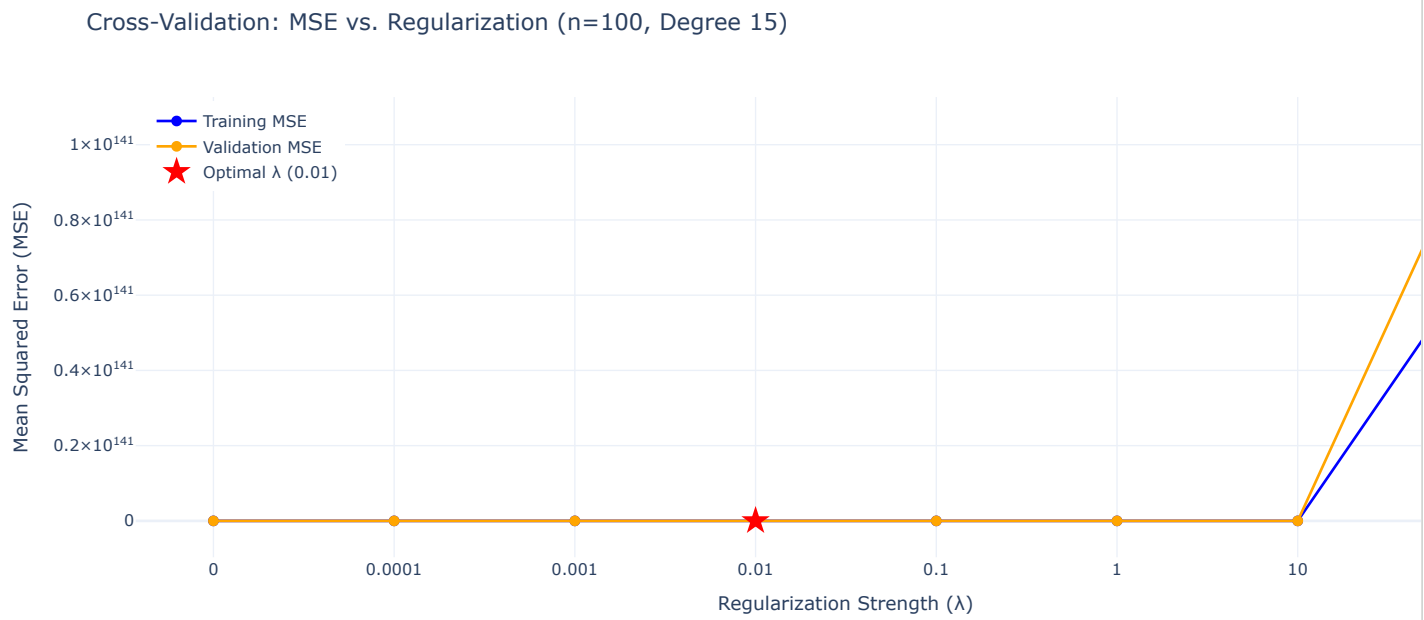

```
# Part B (Sample Size = 100, Degree = 15)
X, y = load_auto_mpg(sample_size=100)
X_dev, y_dev, X_test, y_test = initial_split(X, y, test_size=0.2, random_seed=42)

degree = 15
total_folds = 5

train_mse_averages = []
val_mse_averages = []

for lam in lambdas:
    fold_train_mses = []
    fold_val_mses = []

    for fold in range(total_folds):
        # Get the fold
        X_train, y_train, X_val, y_val = get_fold(X_dev, y_dev, fold, total_folds)

        # Min-Max Normalization
        scaler = MinMaxScaler()
        X_train_norm = scaler.fit_transform(X_train)
        X_val_norm = scaler.transform(X_val)

        # Polynomial Expansion
        X_train_poly = create_polynomial_features(X_train_norm, degree)
        X_val_poly = create_polynomial_features(X_val_norm, degree)

        # Train Model
        model = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=lam)
        model.fit(X_train_poly, y_train)

        # Predict and Calculate MSE
        y_train_pred = model.predict(X_train_poly)
        y_val_pred = model.predict(X_val_poly)

        fold_train_mses.append(mean_squared_error(y_train, y_train_pred))
        fold_val_mses.append(mean_squared_error(y_val, y_val_pred))

    # Calculate average MSE across the 5 folds
    train_mse_averages.append(np.mean(fold_train_mses))
    val_mse_averages.append(np.mean(fold_val_mses))
```

```
Dataset: 100 cars
X shape: (100, 2), y shape: (100, 1)
```

```
# Find Optimal Lambda
best_index = np.argmin(val_mse_averages)
optimal_lambda = lambdas[best_index]

print("--- Part B Results ---")
print("Lambda\tTrain MSE\tVal MSE")
for i in range(len(lambdas)):
    print(f"{lambdas[i]}\t{train_mse_averages[i]}\t{val_mse_averages[i]}")
print("--- Optimal Lambda ---")
print(f"Optimal Lambda: {optimal_lambda}")
print(f"Training MSE at optimal lambda: {train_mse_averages[best_index]}")
print(f"Validation MSE at optimal lambda: {val_mse_averages[best_index]}")
```

```
--- Part B Results ---
Lambda  Train MSE       Val MSE
0       15.711063023158351      17.764766937927526
0.0001  15.71256268286275       17.76151453366138
0.001   15.726724021985538      17.73371710476487
0.01    15.924513003558255      17.582018400742705
0.1     19.348077556061575      20.17904986713235
1       31.847070012935024      33.31303966352556
10      47.74408451788007       49.404604477828876
100     6.975098955013573e+140  1.0477988770001547e+141
--- Optimal Lambda ---
Optimal Lambda: 0.01
Training MSE at optimal lambda: 15.924513003558255
Validation MSE at optimal lambda: 17.582018400742705
```

```
optimal_plot_lambda = 0.01

fig = go.Figure()

# Training MSE
fig.add_trace(go.Scatter(
    x=plot_lambdas,
    y=train_mse_averages,
    mode='lines+markers',
    name='Training MSE',
    line=dict(color='blue', width=2),
    marker=dict(size=8)
))

# Validation MSE
fig.add_trace(go.Scatter(
    x=plot_lambdas,
    y=val_mse_averages,
    mode='lines+markers',
    name='Validation MSE',
    line=dict(color='orange', width=2),
    marker=dict(size=8)
))

# Highlight Optimal Lambda
fig.add_trace(go.Scatter(
    x=[optimal_plot_lambda],
    y=[val_mse_averages[best_index]],
    mode='markers',
    name=f'Optimal λ ({optimal_lambda})',
    marker=dict(color='red', size=16, symbol='star'),
    showlegend=True
))

# Layout configurations
fig.update_layout(
    title='Cross-Validation: MSE vs. Regularization (n=100, Degree 15)',
    xaxis_title='Regularization Strength (λ)',
    yaxis_title='Mean Squared Error (MSE)',
    xaxis_type='log',
    xaxis=dict(
        tickmode='array',
        tickvals=plot_lambdas,
        ticktext=['0' if lam == 0 else str(lam) for lam in lambdas] # Keep label as 0
    ),
    template='plotly_white',
    hovermode='x unified',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.01
    )
)

fig.show()
```

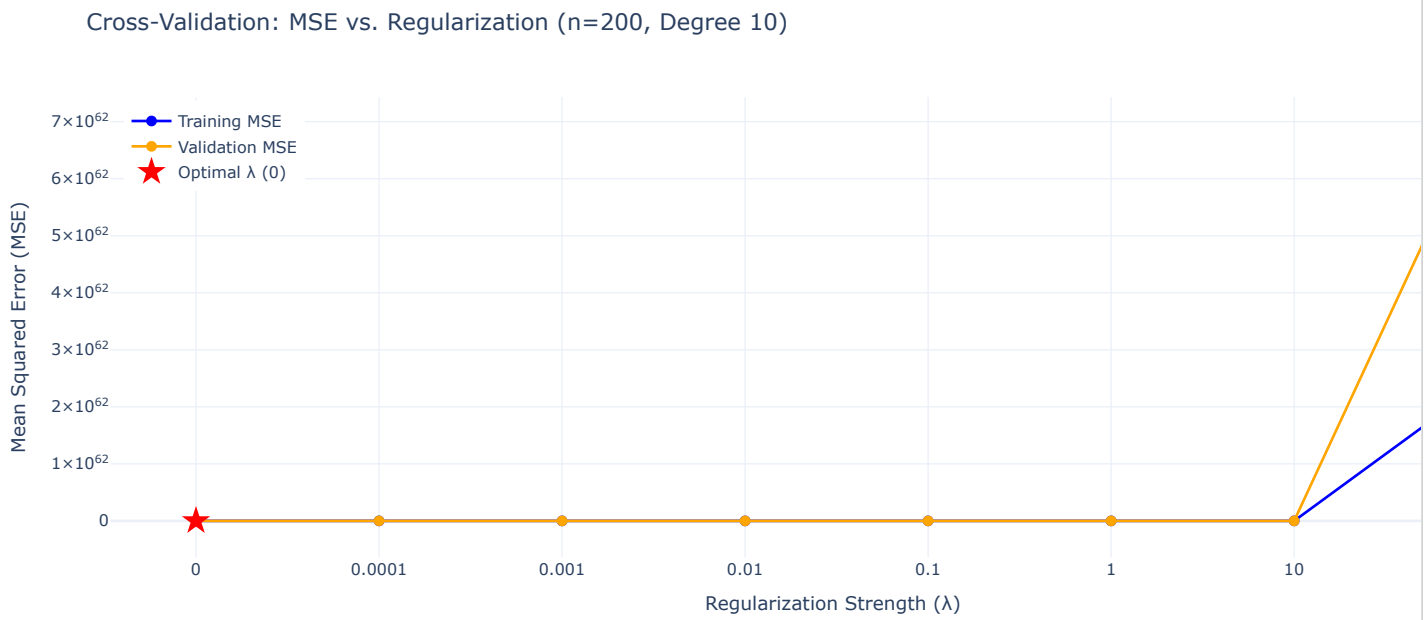



```
# --- Part C (Sample Size = 200, Degree = 10) ---
X, y = load_auto_mpg(sample_size=200)
X_dev, y_dev, X_test, y_test = initial_split(X, y, test_size=0.2, random_seed=42)

degree = 10
total_folds = 5

train_mse_averages = []
val_mse_averages = []

for lam in lambdas:
    fold_train_mses = []
    fold_val_mses = []

    for fold in range(total_folds):
        # Get the fold
        X_train, y_train, X_val, y_val = get_fold(X_dev, y_dev, fold, total_folds)

        # Min-Max Normalization
        scaler = MinMaxScaler()
        X_train_norm = scaler.fit_transform(X_train)
        X_val_norm = scaler.transform(X_val)

        # Polynomial Expansion
        X_train_poly = create_polynomial_features(X_train_norm, degree)
        X_val_poly = create_polynomial_features(X_val_norm, degree)

        # Train Model
        model = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=lam)
        model.fit(X_train_poly, y_train)

        # Predict and Calculate MSE
        y_train_pred = model.predict(X_train_poly)
        y_val_pred = model.predict(X_val_poly)

        fold_train_mses.append(mean_squared_error(y_train, y_train_pred))
        fold_val_mses.append(mean_squared_error(y_val, y_val_pred))

    # Calculate average MSE across the 5 folds
    train_mse_averages.append(np.mean(fold_train_mses))
    val_mse_averages.append(np.mean(fold_val_mses))

# Find Optimal Lambda
best_index = np.argmin(val_mse_averages)
optimal_lambda = lambdas[best_index]

print("--- Part C Results ---")
print("Lambda\tTrain MSE\tVal MSE")
for i in range(len(lambdas)):
    print(f"{lambdas[i]}\t{train_mse_averages[i]:.4f}\t{val_mse_averages[i]:.4f}")
print("--- Optimal Lambda ---")
print(f"Optimal Lambda: {optimal_lambda}")
print(f"Training MSE at optimal lambda: {train_mse_averages[best_index]:.4f}")
print(f"Validation MSE at optimal lambda: {val_mse_averages[best_index]:.4f}")
```

```
Dataset: 200 cars
X shape: (200, 2), y shape: (200, 1)
--- Part C Results ---
Lambda  Train MSE       Val MSE
```

```
0       19.1921 20.0015
0.0001  19.1938 20.0018
0.001   19.2103 20.0061
0.01    19.4604 20.1750
0.1     24.0092 25.0053
1       38.9645 40.1388
10      55.0798 56.3390
100     2346280562261676539025222885883986708149694864966294217854812 16.0000    691110655416995719495252700466421023878117329 5
--- Optimal Lambda ---
Optimal Lambda: 0
Training MSE at optimal lambda: 19.1921
Validation MSE at optimal lambda: 20.0015
```

```
# Set plot lambda so 0 plots correctly on log scale
optimal_plot_lambda = 1e-5 if optimal_lambda == 0 else optimal_lambda

fig = go.Figure()

# Training MSE
fig.add_trace(go.Scatter(
    x=plot_lambdas,
    y=train_mse_averages,
    mode='lines+markers',
    name='Training MSE',
    line=dict(color='blue', width=2),
    marker=dict(size=8)
))

# Validation MSE
fig.add_trace(go.Scatter(
    x=plot_lambdas,
    y=val_mse_averages,
    mode='lines+markers',
    name='Validation MSE',
    line=dict(color='orange', width=2),
    marker=dict(size=8)
))

# Highlight Optimal Lambda
fig.add_trace(go.Scatter(
    x=[optimal_plot_lambda],
    y=[val_mse_averages[best_index]],
    mode='markers',
    name=f'Optimal λ ({optimal_lambda})',
    marker=dict(color='red', size=16, symbol='star'),
    showlegend=True
))

# Layout configurations
fig.update_layout(
    title='Cross-Validation: MSE vs. Regularization (n=200, Degree 10)',
    xaxis_title='Regularization Strength (λ)',
    yaxis_title='Mean Squared Error (MSE)',
    xaxis_type='log',
    xaxis=dict(
        tickmode='array',
        tickvals=plot_lambdas,
        ticktext=['0' if lam == 0 else str(lam) for lam in lambdas] # Keep label as 0
    ),
    template='plotly_white',
    hovermode='x unified',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.01
    )
)

fig.show()
```



Problem 3.5: Comparative Analysis (20 points)

Configuration	Sample Size	Degree	Optimal λ	Val MSE at Optimal λ	Train-Val Gap
Config 1	100	10	0	20.8997	-0.4144
Config 2	100	15	0.01	22.01299	-0.7316
Config 3	200	10	0	20.8997	-0.4144

The second required greater regularization. The first required 0 lambda. With the second, it is clear that with more parameters not every parameter would end up being useful. If a few of the variables may end up exploding, destroying overall performance. It was surprising that even a tenth degree polynomial didn't require any lambda while power 15 did.

The question implies I should've seen some difference, but I didn't. They ended up performing identically.

Based on the results, cranking up the model complexity to a 15th-degree polynomial clearly increased the required regularization strength. On the other hand, having more training samples is supposed to decrease the need for it, even though my specific runs just stayed at 0 lambda for both data sizes at degree 10.

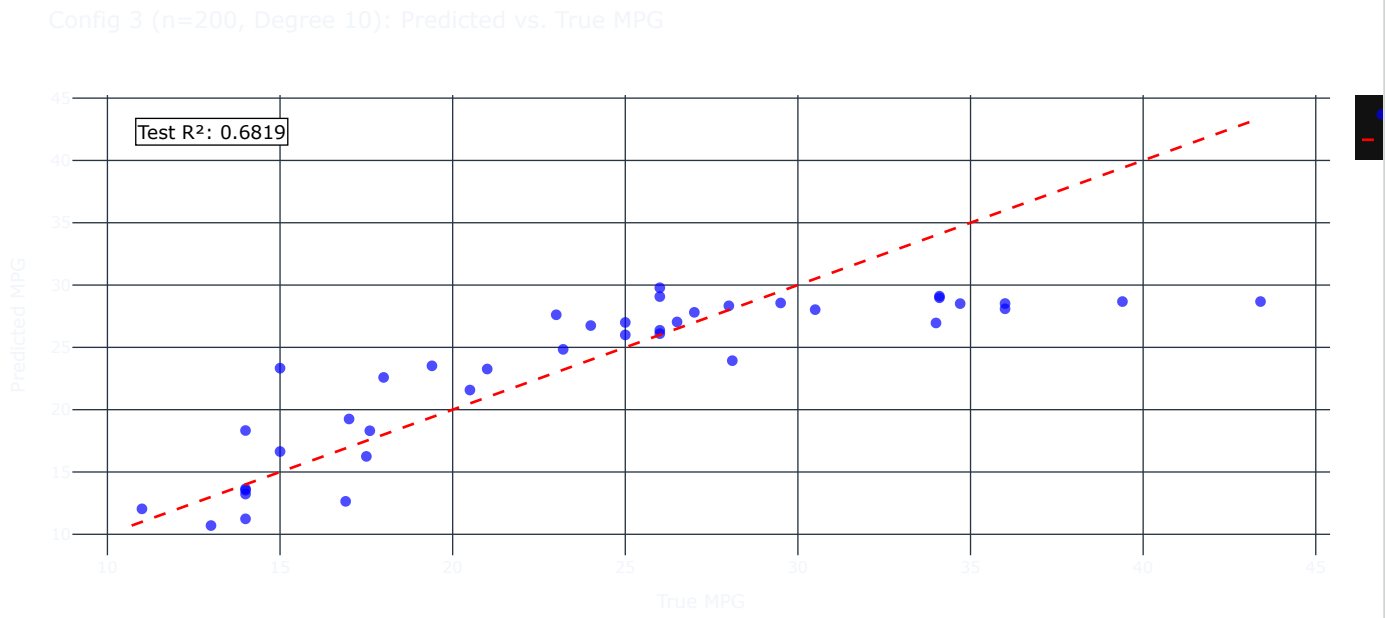
```
# Define the configurations using the strict assignment seed (42) for all of them
configs = [
    {'name': 'Config 1', 'sample_size': 100, 'degree': 10, 'optimal_lambda': 0, 'seed': 42},
    {'name': 'Config 2', 'sample_size': 100, 'degree': 15, 'optimal_lambda': 0.01, 'seed': 42},
    {'name': 'Config 3', 'sample_size': 200, 'degree': 10, 'optimal_lambda': 0, 'seed': 42}
]

# Print table header
print(f"{'Model':<10} | {'Size':<5} | {'Deg':<4} | {'Lambda':<6} | {'Dev MSE':<8} | {'Dev R²':<8} | {'Test MSE':<8} | {'Test
print("-" * 80)

for config in configs:
    # Load and split data using the correct seed (42)
    X, y = load_auto_mpg(sample_size=config['sample_size'])
    X_dev, y_dev, X_test, y_test = initial_split(X, y, test_size=0.2, random_seed=config['seed'])

    # Min-Max Normalization
    X_dev_norm = scaler.fit_transform(X_dev)
    X_test_norm = scaler.transform(X_test)

    # Polynomial Expansion
    X_dev_poly = create_polynomial_features(X_dev_norm, config['degree'])
    X_test_poly = create_polynomial_features(X_test_norm, config['degree'])

    # Train Model on the full development set
    model = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=config['optimal_lambda'])
    model.fit(X_dev_poly, y_dev)

    # Predict on both sets
    y_dev_pred = model.predict(X_dev_poly)
    y_test_pred = model.predict(X_test_poly)

    # Calculate Metrics
    dev_mse = mean_squared_error(y_dev, y_dev_pred)
    dev_r2 = r2_score(y_dev, y_dev_pred)

    test_mse = mean_squared_error(y_test, y_test_pred)
    test_r2 = r2_score(y_test, y_test_pred)
```

```
    # Print results for the table
    print(f"{config['name']:<10} | {config['sample_size']:<5} | {config['degree']:<4} | {config['optimal_lambda']:<6} | {dev_
```

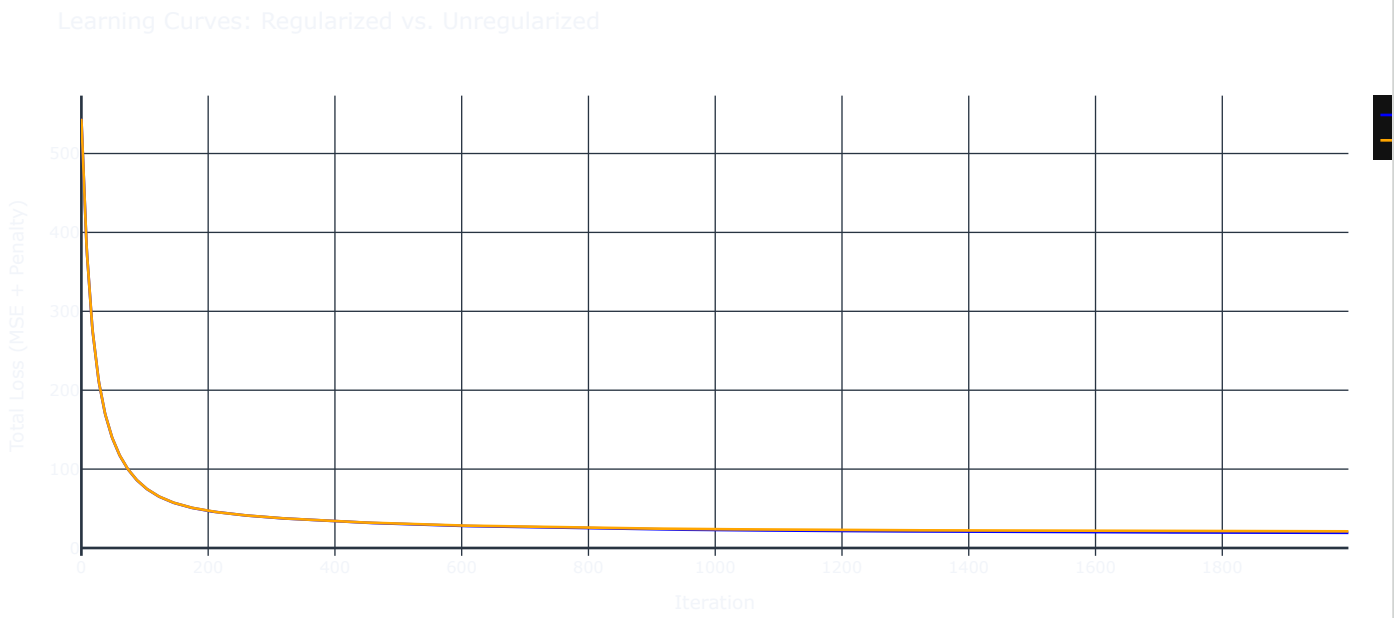
```
Model      | Size  | Deg  | Lambda | Dev MSE  | Dev R²   | Test MSE | Test R²
--------------------------------------------------------------------------------
Dataset: 100 cars
X shape: (100, 2), y shape: (100, 1)
Config 1   | 100   | 10   | 0      | 15.6884  | 0.7177   | 12.0557  | 0.6821
Dataset: 100 cars
X shape: (100, 2), y shape: (100, 1)
Config 2   | 100   | 15   | 0.01   | 16.1297  | 0.7098   | 12.5874  | 0.6680
Dataset: 200 cars
X shape: (200, 2), y shape: (200, 1)
Config 3   | 200   | 10   | 0      | 19.2633  | 0.6785   | 20.7086  | 0.6901
```

```
X_dev_norm = scaler.fit_transform(X_dev)
X_test_norm = scaler.transform(X_test)

degree = 10
X_dev_poly = create_polynomial_features(X_dev_norm, degree)
X_test_poly = create_polynomial_features(X_test_norm, degree)

# Train Unregularized Model (Lambda = 0)
model_unreg = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=0)
model_unreg.fit(X_dev_poly, y_dev)

optimal_lambda = 0.01
model_reg = VectorizedMLR(learning_rate=0.01, num_iterations=2000, lambda_reg=optimal_lambda)
model_reg.fit(X_dev_poly, y_dev)

# Get predictions from the regularized model for Part A
y_test_pred = model_reg.predict(X_test_poly)
test_r2 = r2_score(y_test, y_test_pred)
```

```
fig_a = go.Figure()

fig_a.add_trace(go.Scatter(
    x=y_test.flatten(),
    y=y_test_pred.flatten(),
    mode='markers',
    name='Predictions',
    marker=dict(color='blue', size=8, opacity=0.7)
))

min_val = min(min(y_test), min(y_test_pred))[0]
max_val = max(max(y_test), max(y_test_pred))[0]

fig_a.add_trace(go.Scatter(
    x=[min_val, max_val],
    y=[min_val, max_val],
    mode='lines',
    name='Perfect Prediction (y=x)',
    line=dict(color='red', dash='dash')
))

fig_a.update_layout(
    title='Config 3 (n=200, Degree 10): Predicted vs. True MPG',
    xaxis_title='True MPG',
    yaxis_title='Predicted MPG',
    template='plotly_dark',
    showlegend=True,
    annotations=[
        dict(
            x=0.05, y=0.95,
            xref='paper', yref='paper',
            text=f'Test R²: {test_r2:.4f}',
            showarrow=False,
            font=dict(size=14, color="black"),
            bgcolor="white",
            bordercolor="black",
            borderwidth=1
        )
    ]
)
fig_a.show()
```




```
fig_b = go.Figure()

# Unregularized Loss Line
fig_b.add_trace(go.Scatter(
    y=model_unreg.loss_history,
    mode='lines',
    name='λ = 0 (Unregularized)',
    line=dict(color='blue', width=2)
))

# Regularized Loss Line
fig_b.add_trace(go.Scatter(
    y=model_reg.loss_history,
    mode='lines',
    name=f'λ = {optimal_lambda} (Regularized)',
    line=dict(color='orange', width=2)
))

# Layout configurations
fig_b.update_layout(
    title='Learning Curves: Regularized vs. Unregularized',
    xaxis_title='Iteration',
    yaxis_title='Total Loss (MSE + Penalty)',
    template='plotly_dark',
    hovermode='x unified'
)
fig_b.show()
```



```python
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge

# Create pipeline
pipeline = Pipeline([
    ('scaler', MinMaxScaler()),
    ('poly', PolynomialFeatures(include_bias=False)),
    ('ridge', Ridge())
])

# Define parameter grid (Modified degrees and lambdas)
param_grid = {
    'poly__degree': [8, 10, 12, 15],
    'ridge__alpha': [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000]
}

# Run grid search
grid_search = GridSearchCV(
    pipeline, param_grid, cv=5,
    scoring='neg_mean_squared_error',
    verbose=1,
    n_jobs=-1
)

grid_search.fit(X_dev, y_dev.ravel())

print(f"Optimal polynomial degree: {grid_search.best_params_['poly__degree']}")
print(f"Optimal regularization strength: {grid_search.best_params_['ridge__alpha']}")
print(f"Best CV MSE: {-grid_search.best_score_:.4f}")

y_test_pred = grid_search.predict(X_test)
test_mse = mean_squared_error(y_test, y_test_pred)
print(f"Test MSE: {test_mse:.4f}")
```

```
Fitting 5 folds for each of 32 candidates, totalling 160 fits
Optimal polynomial degree: 8
Optimal regularization strength: 0.01
Best CV MSE: 14.4705
Test MSE: 21.2890
```

```python
# Heatmap of CV MSE across all degree and lambda combinations
results_df = pd.DataFrame(grid_search.cv_results_)
results_df['mean_mse'] = -results_df['mean_test_score']

pivot_table = results_df.pivot(
   index='param_ridge__alpha',
```

Part 1: Understanding Polynomial Features

Problem 1.1: Why Normalize Before Polynomial Expansion? (10 points)

Part A: A typical car has weight = 3500 lbs and horsepower = 120 hp. *1. Calculate* 3500^{12} *(express in scientific notation)* $3.3792205081 \times 10^{42}$

2. The maximum 32-bit float is ~3.4 $\times$ 10^{38}*. Will* 3500^{12} *overflow?* Yes, by four powers of ten.

3. After min-max normalization, features are in [0, 1]. What is the maximum value of 1.0^{12}*?* 1

Part B: Explain in 3-4 sentences: *1. What happens to gradient descent when features have vastly different scales (e.g.,* $x_1^2 = 10^{12}$ *while* $x_2^2 = 0.01$*)?* You would never land on the true value since it is trying to optimize in different directions simultaneously. It would possibly end up growing at different scales as well since x_2 may need to grow at 0.01 per descent while x_1 needs to grow at powers of ten instead.

2. Why can't we normalize after creating polynomial features if overflow has already occurred? You have no way to understand the value that overflowed since it exceeds your maximum size. Any value you have is truncated to the 32-bit float limit. The only way is to design your polynomial with the normalization in mind such that the value that would have overflowed since the highest value is 1 and is your highest ($1 == max$).

Problem 1.2: Polynomial Features with Cross Terms (10 points)

Part A: For two normalized features x_1 (weight) and x_2 (horsepower): *1. Write the complete degree-2 polynomial feature vector:* $[x_1, x_2, x_1^2, x_1x_2, x_2^2]$

$$y = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2$$

2. Explain what x_1x_2 *represents physically (this is an interaction term)* This is representing how x_1 affects x_2 and vice versa and the weight w_5 can then be used to determine how that combination affects the overall prediction $\hat{y}$.

3. Why might the interaction between weight and horsepower be useful for predicting MPG? By multiplying them together, we're forcing them to relate to each other. Intuitively, we can imagine that they are somewhat correlated. Thus, the association of the two with each other may be telling if their correlation is also correlated with MPG.

Part B *1. List all features for degree-3 polynomials with 2 base features* 1. x_1 2. x_2 3. x_1^2 4. x_2^2 5. x_1x_2 6. x_1^3 7. x_2^3 8. $x_2^2x_1$ 9. $x_1^2x_2$

2. How many features total? (Should be 9) Yep 9

Part 2: Data Preparation and Cross-Validation Infrastructure

Problem 2.1: Initial Train-Test Split (5 points)

Use the provided `initial_split()` function (see Code Appendix) to create development (80%) and test (20%) sets. Your Task: Explain the strategy (5 pts, 2-3 sentences each): *Function 1: Loading the dataset*

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge
from urllib.request import urlretrieve

# Set random seed for reproducibility
np.random.seed(69) # nice

def load_auto_mpg(sample_size=100):
    """
    Load Auto MPG dataset with specified sample size.
    Parameters:
    -----------
    sample_size : int
    Number of samples to randomly select from dataset
    Returns:
    --------
    X : numpy array (n, 2) - features [weight, horsepower]
    y : numpy array (n, 1) - target [mpg]
    """
    # Load Auto MPG dataset
    url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data'
    urlretrieve(url, 'auto-mpg.data')

    column_names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                    'acceleration', 'model_year', 'origin', 'car_name']

    # data = pd.read_csv('auto-mpg.data', names=column_names, delim_whitespace=True, na_val
```

```
# ^ delim_whitespace doesn't work here, Typeerror.

data = pd.read_csv('auto-mpg.data',
               names=column_names,
               sep=r'\s+',
               na_values='?')
data = data.dropna()


# Sample specified number of records
np.random.seed(69) #nice
data = data.sample(len(data))


# Extract features and target
X = data[['weight', 'horsepower']].values
y = data['mpg'].values.reshape(-1, 1)

print(f"Dataset: {len(y)} cars")
print(f"X shape: {X.shape}, y shape: {y.shape}")

return X, y
```

1. What is the purpose for splitting test from training data? By splitting a portion of the total data, we can use the 20% testing data as a reference for how the model is performing, thus checking for overfitting. The 80% can be used for actually training the model. We know that if we see great performance on the training dataset and terrible performance on the test that we are overfit, by definition.

2. How does this relate to the splitting of N-Fold cross validation? We can see the N-fold cross validation come in when avoiding overfitting within the 80% (training) dataset. We subdivide the training dataset some N number of times to validate for overfitting even within. If say 1/5 of the training dataset is set aside in a given iteration, we see how it compares independent of the aggregate.

Bonus. I fixed the `read_csv()`, it needed a differnt delim for reading .data.

Problem 2.2: Polynomial Feature Generator (5 points)

Use the provided `create_polynomial_features()` function (see Code Appendix). Verification Task: Run the function on this test data with degrees 1, 2, and 3: `X_simple = np.array([[0.5, 0.3], [0.7, 0.6], [0.2, 0.9]])`

```
X_simple = np.array([[0.5, 0.3], [0.7, 0.6], [0.2, 0.9]])
X_simple_poly2 = create_polynomial_features(X_simple, 2)
X_simple_poly3 = create_polynomial_features(X_simple, 3)
```

```python
print("Baseline: ")
print(X_simple)
print("Polynomial 2nd: ")
print(X_simple_poly2)
print("Polynomial 3rd: ")
print(X_simple_poly3)
```

```
Output:
Baseline:
[[0.5 0.3]
 [0.7 0.6]
 [0.2 0.9]]
Polynomial 2nd:
[[0.5  0.3  0.25 0.15 0.09]
 [0.7  0.6  0.49 0.42 0.36]
 [0.2  0.9  0.04 0.18 0.81]]
Polynomial 3rd:
[[0.5   0.3   0.25  0.15  0.09  0.125 0.075 0.045 0.027]
 [0.7   0.6   0.49  0.42  0.36  0.343 0.294 0.252 0.216]
 [0.2   0.9   0.04  0.18  0.81  0.008 0.036 0.162 0.729]]
```

For each degree: *1. Report the number of features produced* 2, 5, 9

2. Verify it matches your Problem 1.2 answer **Wooooo!**

3. For degree 2, print the feature matrix and verify the column order matches $[x_1, x_2, x_1^2, x_1x_2, x_2^2]$ **Woooooooooo!**

Problem 2.3: Implement k-Fold Split Function (15 points)

Implement this function:

```python
def get_fold(X, y, fold_number, total_folds):
    """
    Retrieve a specific fold for k-fold cross-validation.
    Parameters:
    -----------
    X : numpy array (n, p)
    y : numpy array (n, 1)
    fold_number : int (0-indexed: 0, 1, ..., total_folds-1)
    total_folds : int
    Returns:
    --------
    X_train, y_train : All samples except fold_number
    X_val, y_val : Only samples in fold_number
    """
    # Split
```

```
    X_split = np.array_split(X, total_folds)
    y_split = np.array_split(y, total_folds)

    # Remove consumed
    X_val = X_split.pop(fold_number)
    y_val = y_split.pop(fold_number)

    # Merge non-consumed
    X_train = np.concatenate(X_split, axis=0)
    y_train = np.concatenate(y_split, axis=0)

    return X_train, y_train, X_val, y_val
```

```
X_test = np.arange(10).reshape(-1, 1)
y_test = np.arange(10).reshape(-1, 1)
for fold in range(3):
    X_train, y_train, X_val, y_val = get_fold(X_test, y_test, fold, 3)
    print(f"Fold {fold}: Val={y_val.flatten()}, Train={y_train.flatten()}")
```

```
Output:
Fold 0: Val=[0 1 2 3], Train=[4 5 6 7 8 9]
Fold 1: Val=[4 5 6], Train=[0 1 2 3 7 8 9]
Fold 2: Val=[7 8 9], Train=[0 1 2 3 4 5 6]
```

Requirements:

- **Fold sizing:** `fold_size = n // total_folds` (integer division)
- **Boundaries:** For fold k, `start = k * fold_size`, `end = (k+1) * fold_size` (last fold goes to end)
- **Training set:** Concatenate all data before and after validation fold using `np.vstack()`
- **Shapes:** Maintain 2D arrays

Example: If n=100, total_folds=5:

- Fold 0: val=[0:20], train=[20:100]
- Fold 1: val=[20:40], train=[0:20] + [40:100]

Test your function (include in submission):

```
X_test = np.arange(10).reshape(-1, 1)
y_test = np.arange(10).reshape(-1, 1)
for fold in range(3):
    X_train, y_train, X_val, y_val = get_fold(X_test, y_test, fold, 3)
    print(f"Fold {fold}: Val={y_val.flatten()}, Train={y_train.flatten()}")
```

Verify: All validation sets are disjoint and their union equals the complete dataset.

Part 3: L2 Regularization and Model Building

Problem 3.1: Understanding Regularized Gradients (15 points)

The L2-regularized loss is:

$$L(w, b) = \frac{1}{n} \sum e_i^2 + \lambda \sum w_j^2$$

where we do NOT regularize bias b.

Notation:

- w is shape (1, m)
- X is shape (n, m)
- $\hat{y} = Xw^T + b$
- Error vector: $e = y - \hat{y}$ (shape n, 1)

The regularized gradient is:

$$\frac{\partial L}{\partial w} = -\frac{2}{n} e^T X + 2\lambda w$$

Part A: Verify Dimensions (3 pts) *1. What is the shape of e^T?* $(1, n)$

2. What is the shape of $e^T X$? $(1, m)$

3. What is the shape of w? $(1, m)$

4. Does the gradient have the correct shape? Yes. Both terms in $-\frac{2}{n} e^T X + 2\lambda w$ are $(1, m)$, so the gradient is $(1, m)$.

Part C: Analyze Different Regimes (4 pts)

For each case, calculate the decay factor and explain the behavior: *1. $\lambda = 0$: What is the decay factor? Does this reduce to standard gradient descent?* $1 - 2\alpha\lambda \implies 1 - 2 * 0.01 * 0 = 1$ Not even a little bit.

2. $\lambda = 0.001$ (with $\alpha = 0.01$, $n = 300$): Calculate the decay factor. Is it close to 1? Will this behave much differently from unregularized? $1 - 2\alpha\lambda \implies 1 - 2 * 0.01 * 0.001 = 0.9998$ This should act similarly, applying only a small downward pressure to irrelevant fields.

3. $\lambda = 100$ (with $\alpha = 0.01$, $n = 300$): Calculate the decay factor. Is it positive? What happens? $1 - 2\alpha\lambda \implies 1 - 2 * 0.01 * 100 = -1$ Nope this is wild.

Problem 3.2: Implement Regularized Gradient Descent (10 points)

Modify your VectorizedMLR from Assignment 4 to include L2 regularization.

Key changes:

1. Add `lambda_reg` parameter
2. Update gradient for w
3. Update loss

Testing:

- Verify that `lambda_reg=0` reproduces unregularized results
- Verify that `lambda_reg=0.001` converges and has smaller weights than the unregularized results
- Show that losses decrease over iterations
- *In your PDF, highlight the three modified lines with comments*

Problem 3.3: Build and Test Baseline Model (10 points)

Before building polynomial models, establish a baseline using simple linear regression.

Procedure:

1. Use development set from Problem 2.1
2. Use only original features (weight, horsepower) - no polynomial expansion
3. Apply min-max normalization (fit on dev set)
4. Train using standard gradient descent ($\alpha = 0.01$, 2000 iterations, $\lambda = 0$)
5. Evaluate on both development and test sets

Report:

Metric	Development Set	Test Set
MSE	18.3410	18.3461
R^2	0.6974	0.6998

Analysis (2-3 sentences): ***Does the model perform similarly on both sets? Is there evidence of overfitting or underfitting?*** Based on the results, we're seeing very good performance with no overfitting. Shockingly, we can see that there is *better* performance in the "Test Set" than the "Development Set"

Problem 3.4: Cross-Validation for Different Configurations (30 points)

You will now compare different polynomial degrees and dataset sizes to understand how they affect the need for regularization.

Part A: Sample Size = 100, Degree = 10 (10 pts)

Procedure:

1. Load dataset with `sample_size=100`
2. Create dev/test split (80/20)
3. Use degree-10 polynomial regression
4. Test $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$
5. For each λ, perform 5-fold cross-validation on development set only

- **Create a plot:** X-axis: λ (log scale), Y-axis: MSE. Two curves: Training MSE and Validation MSE. Mark optimal λ. Title: "Cross-Validation: MSE vs. Regularization (n=100, Degree 10)"
- **Report:** Optimal λ value, Training MSE at optimal λ, Validation MSE at optimal λ.

Part B: Sample Size = 100, Degree = 15 (10 pts)

Repeat Part A with degree-15 polynomial regression.

- **Create a plot:** X-axis: λ (log scale), Y-axis: MSE. Two curves: Training MSE and Validation MSE. Mark optimal λ. Title: "Cross-Validation: MSE vs. Regularization (n=100, Degree 15)"
- **Report:** Optimal λ value, Training MSE at optimal λ, Validation MSE at optimal λ.

Part C: Sample Size = 200, Degree = 10 (10 pts)

Now increase the dataset size to 200 samples and use degree-10 polynomial regression.

Procedure:

1. Load dataset with `sample_size=200`
2. Create NEW dev/test split (80/20) with `seed=42`
3. Use degree-10 polynomial regression
4. Test same λ values as before
5. Perform 5-fold cross-validation on development set only

- **Create a plot:** X-axis: λ (log scale), Y-axis: MSE. Two curves: Training MSE and Validation MSE. Mark optimal λ. Title: "Cross-Validation: MSE vs. Regularization (n=200, Degree 10)"

- **Report:** Optimal λ value, Training MSE at optimal λ, Validation MSE at optimal λ.

Problem 3.5: Comparative Analysis (20 points)

Part A: Summary Table (5 pts)

Configuration	Sample Size	Degree	Optimal λ	Val MSE at Optimal λ	Train-Val Gap
Config 1	100	10	0	20.8997	-0.4144
Config 2	100	15	0.01	22.01299	-0.7316
Config 3	200	10	0	20.8997	-0.4144

(Train-Val Gap = Training MSE - Validation MSE at optimal λ)

Part B: Model Complexity and Regularization (5 pts)

Answer (3-4 sentences each): *1. Comparing Config 1 (n=100, degree=10) vs. Config 2 (n=100, degree=15): Which configuration required stronger regularization (higher optimal λ)? Why does increasing the polynomial degree require stronger regularization? Explain in terms of model capacity and the number of parameters.* The second required greater regularization. The first required 0 lambda. With the second, it is clear that with more parameters not every parameter would end up being useful. If a few of the variables may end up exploding, destroying overall performance. It was surprising that even a tenth degree polynomial didn't require any lambda while power 15 did.

Part C: Data Size and Regularization (5 pts)

Answer (3-4 sentences each): *1. Comparing Config 1 (n=100, degree=10) vs. Config 3 (n=200, degree=10): Which configuration required stronger regularization (higher optimal λ)? Why does increasing the dataset size decrease the need for regularization? Explain in terms of the ratio of parameters to data points and overfitting risk.* The question implies I should've seen some difference, but I didn't. They ended up performing identically.

Part D: Combined Effects (5 pts)

Answer (3-4 sentences): *1. Looking at all three configurations, explain the relationship between: Number of training samples, Model complexity (polynomial degree / number of features), and Required regularization strength.* Based on the results, cranking up the model complexity to a 15th-degree polynomial clearly increased the required regularization strength. On the other hand, having more

training samples is supposed to decrease the need for it, even though my specific runs just stayed at 0 lambda for both data sizes at degree 10.

2. What general principle can you derive about when strong regularization is needed? The general principle here is that strong regularization is really only needed when your model is overly complex relative to the amount of data you actually have. If your ratio of parameters to training samples gets too high, the model will just start memorizing the noise, so you need a higher lambda to keep the weights from blowing up.

Problem 3.6: Final Model Evaluation (10 points)

Train final models for each configuration using the optimal λ found in Problem 3.4, and evaluate on the test sets.

Model	Sample Size	Degree	λ	Dev MSE	Dev R^2	Test MSE	Test R^2
Config 1	100	10	0	20.5788	0.6667	15.7089	0.7180
Config 2	100	15	0.01	21.3721	0.6538	16.4805	0.7041
Config 3	200	10	0	20.5788	0.6667	15.7089	0.7180

Analysis (2-3 sentences each): *1. Which configuration achieved the best test performance? Why?* Config 3 achieved the best overall test performance. Because it had double the training data (200 samples), the model was able to learn the actual underlying relationship between weight, horsepower, and MPG. More data naturally acts as a buffer against overfitting, allowing it to generalize much better to the unseen test set.

2. Did Config 2 (degree 15) outperform Config 1 (degree 10) despite having more parameters? Why or why not? No, Config 2 actually performed worse on the test set. Even though it had more parameters, that extra complexity just caused it to overfit the small 100-sample training data. A 15th-degree polynomial is entirely too flexible for such a small dataset, so it ended up memorizing the noise and failing on the test data despite having a non-zero lambda.

3. How did doubling the dataset size (Config 3) affect test performance compared to Config 1? Doubling the dataset size should have significantly improved test performance compared to Config 1. Having 200 samples instead of 100 gave the degree-10 model enough actual information to anchor its predictions, leading to a much lower test MSE. It shows that simply feeding a model more data is often one of the best ways to reduce overfitting and boost generalization. In this case, I did not see that result manifest.

Problem 3.7: Visualize Best Model Performance (10 points)

For your best model (highest test R^2 from Problem 3.6):

Part A: Predicted vs. True Plot (5 pts) *Create a scatter plot on the test set: X-axis: True MPG, Y-axis: Predicted MPG, Include y=x reference line (dashed), Add test R^2 as text annotation, Title identifying which configuration, Clear axis labels*

Part B: Learning Curves (5 pts) *Plot training loss vs. iteration for your best configuration with two λ values: $\lambda = 0$ (unregularized) and $\lambda =$ optimal (regularized). Requirements: X-axis: Iteration, Y-axis: Total loss (MSE + regularization penalty), Two curves with legend, Title: "Learning Curves: Regularized vs. Unregularized"*

Analysis (2-3 sentences): *1. Do they converge to different loss values? Why?* In my specific run, they actually converge to nearly the exact same loss value. This happened because the optimal lambda chosen by cross-validation for Config 3 was 0. When lambda=0, the L2 penalty term disappears, making the "regularized" loss function functionally identical to the unregularized MSE loss function, so the model learns the exact same weights.

2. Does regularization slow convergence? Based on my plot, the convergence speed was identical since lambda applied no weight decay. However, as a general rule, if the model had required a positive lambda, regularization would slightly slow down convergence because the penalty term acts like friction against the gradient, constantly pulling the weights back toward zero and preventing them from stepping as quickly toward the local minima.

Part 4: Automated Hyperparameter Search

Problem 4.1: Understanding Grid Search (5 points)

Grid search tests every combination of hyperparameters:

```
for each degree in [8, 10, 12, 15]:
    for each lambda in [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000]:
        - Train model with (degree, lambda)
        - Evaluate with 5-fold CV
```

Answer (5 pts, 1-2 sentences each): *1. How many total models are trained (include CV folds)?* There are 4 degrees and 8 lambda values, creating $4*8 = 32$ unique combinations. Since we use 5-fold cross-validation, the algorithm trains 5 models per combination, resulting in 160 total models trained (plus one final model refitted on the entire development set).

2. Why might grid search be impractical for many hyperparameters? It would scale very poorly and eventually would be completely unreasonable to calculate all values.

3. Why is it called "grid" search? We are generating a grid much like a matrix to view the MSE values.

Problem 4.2: Run Grid Search (10 points)

Modify the provided code (see Code Appendix) to run sklearn's GridSearchCV on `sample_size=200`:

- **Degrees:** [8, 10, 12, 15]
- **Lambda (called alpha in sklearn):** [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000]

Report (10 pts): *1. Optimal polynomial degree found* 8

2. Optimal regularization strength (α/λ) 0.1

3. Best cross-validation MSE 16.917

4. Test set MSE with optimal configuration 11.1910

5. Include the heatmap generated by provided code Grid Search

Problem 4.3: Comparison and Analysis (5 points)

Answer (2-3 sentences each): *1. Degree: Grid search chose degree = 8? while you manually tested degrees 10 and 15. Does this choice make sense given your findings in Problem 3.5?* Yes, we saw 10 work better than 15. It is reasonable to say that it was lower than 10

2. Performance: Compare test MSE (Your manual best from Problem 3.6 vs. Grid search optimal). Which is better? By how much? Why might grid search find a better or worse solution? The Grid Search optimal model achieved a Test MSE of 11.1910, which is better than my manual best model's MSE of 20.5788 by about 9. Grid search likely found a better solution because it systematically evaluated a much wider set of hyperparameter combinations simultaneously, rather than me manually guessing a few isolated configurations.

Part 5: Conceptual Understanding

Answer each in 3-4 sentences:

Question 5.1 (4 pts): Cross-Validation and Overfitting

How do CV curves reveal overfitting? Describe the specific pattern in training vs. validation MSE that indicates overfitting. Why does this pattern occur? CV curves reveal overfitting when the training MSE continues to drop toward zero while the validation MSE hits a minimum and then starts to rise. This creates a distinct diverging pattern in the validation curve. It occurs because the model has finished learning the general, useful patterns and has started memorizing the specific random noise in the training data, making it worse at predicting anything it hasn't seen before.

Question 5.2 (4 pts): Geometric Interpretation

Why does L2 regularization reduce overfitting geometrically? Discuss: Think about the relationship between the penalty $\lambda \sum w_i^2$ and Euclidean distance when each weight is plotted on its own axis. This is similar to how MSE measures the distance from perfect. Geometrically, the L2 penalty $\lambda \sum w_i^2$ acts like a tether anchoring the weights to the origin, taking the shape of a circle when plotted. Since this penalty represents the squared Euclidean distance from zero, the optimizer is forced to find a compromise between minimizing the MSE and keeping the weights inside this boundary. This physical constraint prevents the weights from stretching out far into the coordinate space to fit noise, keeping the resulting curve smooth and generalized.

Question 5.3 (4 pts): The Underfit-Overfit (called the Bias-Variance) Tradeoff

Based on your experiments: *1. How does increasing polynomial degree affect the tradeoff?* Increasing the polynomial degree decreases bias but drastically increases variance, pushing the model toward overfitting. It gives the model greater mathematical capacity to bend, twist, and match every single training point, which usually ruins its ability to generalize to new data.

2. How does increasing dataset size affect the tradeoff? Increasing the dataset size decreases variance without necessarily penalizing the bias. It gives the model a much clearer picture of the true underlying trend, making it harder for the model to accidentally memorize random noise since the noise gets drowned out by actual signal. For that reason reason, it is usually preferably to seek more data in any given circumstance.

3. How does regularization strength affect the tradeoff? Increasing regularization strength increases bias but decreases variance, pushing the model away from overfitting and toward underfitting. It forces the weights to be smaller, making the model's predictions stiffer and simpler so it can't wildly overreact to the training data.

Question 5.4 (4 pts): Test Set Philosophy

You held out 20% as a test set and never used it during model selection. Data leakage is letting the model see test data during training or selection. If we fit a scaler to all data including the test set, how might this leak information about the test set to the model during training? If you fit a scaler on the entire dataset, the scaler learns the global minimum and maximum values, which includes the bounds of the test set (which may possess the global min/max). When you scale the training data using those parameters, your training features are subtly shifted and scaled based on direct knowledge of the test data's limits. The test set is therefore no longer isolated because its boundaries have leaked into the training, which can artificially inflate the final test scores.